Reporte: Consistencia Multi-Dispositivo en WiFi — Crozzo POS

> Fecha: 2026-11-08 · Objetivo: analizar cómo se comunican 2-5 dispositivos en la misma red, cuánto se transmite, y cómo se evita duplicación o aplicación múltiple de datos.

---

1. El problema que describes (y es real)

> "Si 20 dispositivos están en la misma red, ¿una comanda se aplica 20 veces? ¿Se duplica? ¿Se copia en todos?"

Respuesta corta: NO, si el sistema está bien implementado. Pero hay riesgos latentes.

---

2. Mecanismos de deduplicación que YA existen

2.1 transaction_id (UUID único por comanda)

Cada comanda tiene un `transaction_id` (UUID v4) generado al crearla:

```javascript
// CrozzoPosMain.js ~10074
transaction_id: newCrozzoComandaTransactionId(),  // UUID v4
```

Cómo evita duplicación:
```
Dispositivo A crea comanda → transaction_id = "abc-123"
Dispositivo A envía comanda a B, C, D
Dispositivo B recibe → busca comanda con transaction_id "abc-123"
  → Si ya existe → ignora (ya la tiene)
  → Si no existe → la agrega
```

Código:
```javascript
// CrozzoComandasCloudSync.js:665
function findComandaForCloudPay(pay, row) {
  var tid = String(pay.transaction_id || (row && row.id) || '').trim();
  var list = global.comandas || [];
  if (tid) {
    var byTid = list.find(function (c) {
      return c && c.transaction_id && String(c.transaction_id) === tid;
    });
    if (byTid) return byTid;  // ← ya existe, no duplicar
  }
  ...
}
```

2.2 comandaTidRecent (ventana de dedup temporal)

Marca un `transaction_id` como "visto recientemente" por 45-120 segundos:

```javascript
// CrozzoComandasCloudSync.js:281
function comandaTidRecent(tid, maxMs) {
  var e = global.__crozzoComandaTidSeen[String(tid)];
  return !!(e && Date.now() - e.at < (maxMs || 120000));
```

}
```

**Cómo funciona:**
```
Dispositivo A crea comanda → marca tid como "visto" (timestamp)
Dispositivo B recibe comanda → verifica si tid fue visto recientemente
  → Si sí → es echo propio o duplicado → ignora
  → Si no → es nueva → la agrega y marca como visto
```

### 2.3 comandaInsertLikelyOwnEcho (detección de echo propio)

Cuando Supabase Realtime devuelve un INSERT que el mismo dispositivo acaba de pushear, lo detecta y no reaplica:

```javascript
// CrozzoComandasCloudSync.js:1135
function comandaInsertLikelyOwnEcho(row) {
  if (tid && comandaTidRecent(tid, 45000)) return true;  // fue visto hace <45s
  if (findComandaForCloudPay(pay, row)) return true;       // ya existe en lista
  ...
}
```

Esto evita que el dispositivo que envió la comanda la "reciba de vuelta" y la duplique.

2.4 device_id (quién envió)

Cada mensaje incluye `device_id` del emisor. El receptor puede verificar si el mensaje es propio:

```javascript
// En el sync, se verifica si el mensaje viene del mismo dispositivo
if (row.device_id === myDeviceId) → ignorar (echo propio)
```

2.5 Estado de comanda (no se reaplica si ya está en ese estado)

```javascript
// CrozzoComandasCloudSync.js:671
var byTid = list.find(function (c) {
  return c && c.transaction_id && String(c.transaction_id) === tid && c.estado !== 'entregada';
});
```

Si la comanda ya está en estado "entregada", no se reaplica un cambio a "pendiente".

---

3. Flujo real de una comanda entre 5 dispositivos

```
DISPOSITIVO A (mesero) crea comanda
  transaction_id = "abc-123"
  device_id = "tablet-1"
  estado = "pendiente"
        |
        ├──→ Envía a Caja (HTTP/WS LAN o Cloud)
        |       Caja recibe → busca "abc-123" → no existe → la agrega
        |       Caja marca "abc-123" como visto
        |
```
```

```
        ├──► Envía a Cocina (HTTP/WS LAN o Cloud)
        │       Cocina recibe → busca "abc-123" → no existe → la agrega
        │       Cocina marca "abc-123" como visto
        │
        ├──► Envía a Tablet B (HTTP/WS LAN o Cloud)
        │       Tablet B recibe → busca "abc-123" → no existe → la agrega
        │       Tablet B marca "abc-123" como visto
        │
        └──► Envía a Tablet C (HTTP/WS LAN o Cloud)
                Tablet C recibe → busca "abc-123" → no existe → la agrega
                Tablet C marca "abc-123" como visto

RESULTADO: La comanda "abc-123" existe UNA vez en cada dispositivo. No se duplica.
```

¿Qué pasa si el mismo mensaje llega dos veces?

```
Dispositivo A envía comanda "abc-123" a Caja
Caja recibe → la agrega → marca como visto
        │
        │ (red lenta, mensaje se reenvía)
        │
Caja recibe "abc-123" de nuevo → busca → ya existe → IGNORA
```

¿Qué pasa si un dispositivo se desconecta y reconecta?

```
Tablet B se desconecta → no recibe comandas nuevas
Tablet B se reconecta → pull completo (GET /api/comandas?since=...)
        │
        ├──► Recibe comandas que se perdió
        │       → busca por transaction_id → agrega las nuevas
        │       → ignora las que ya tiene
        │
        └──► Recibe comandas que ya tiene
                → busca por transaction_id → ya existe → ignora
```

---

3. Comunicación real entre 2-5 dispositivos en WiFi

3.1 Topología

```
                    ┌──────────────┐
                    │  CAJA (A)    │  ← Servidor LAN (port 3000/3001)
                    │  Rol A       │  ← mDNS anuncia _crozzo-pos._tcp
                    └──────┬───────┘
                           │
                ┌──────────┼──────────┐
                │          │          │
          ┌─────┴─────┐ ┌──┴──────┐ ┌─┴─────────┐
          │ Tablet B  │ │ Tablet C│ │ Cocina D  │
          │ (mesero)  │ │ (mesero)│ │           │
          └───────────┘ └─────────┘ └───────────┘
```

3.2 Flujo de una comanda (paso a paso)

```
1. Mesero en Tablet B toma pedido → crea comanda local
   transaction_id = "abc-123", device_id = "tablet-B"

2. Tablet B envía comanda a Caja (HTTP POST /api/comandas o WS)
   → Caja recibe → agrega → marca como visto → WS broadcast a todos

3. Caja recibe WS broadcast → ya tiene la comanda → ignora (echo propio)
   Cocina recibe WS broadcast → busca "abc-123" → no existe → agrega → imprime
   Tablet C recibe WS broadcast → busca "abc-123" → no existe → agrega → muestra en UI

4. Cocina prepara → marca "preparando" → envía cambio de estado
   → Caja recibe → actualiza comanda "abc-123" → estado "preparando"
   → WS broadcast → Tablet B y C actualizan

5. Cocina marca "listo" → envía cambio de estado
   → Mismo flujo → todos actualizan

6. Mesero entrega → marca "entregada"
   → Mismo flujo → todos actualizan
```

3.3 Volumen real de tráfico (5 dispositivos, 1 hora)

Evento	Frecuencia	Tamaño	Dispositivos	Bytes/hora
Comandas nuevas	6-12/hora	500-1,500	1 → 4	3,000-18,000 × 4 = 12-72 KB
Cambios de estado	6-12/hora	100-200	1 → 4	0.6-2.4 KB × 4 = 2.4-9.6 KB
Sync runtime	Cada 2-4s	2,000-4,000	Bidireccional × 5	1.8-7.2 MB × 5 = 9-36 MB
Presence	Cada 5-10s	80-120	Bidireccional × 5	29-86 KB × 5 = 145-430 KB
Total				**~9-36 MB/hora**

3.4 ¿Es mucho? No.

- WiFi LAN: 50-800 Mbps = 6-100 MB/s
- 9-36 MB/hora = 2.5-10 KB/s
- **Uso del ancho de banda: <0.1%**

---

4. Mecanismos de consistencia (cómo se evita el caos)

4.1 Deduplicación por transaction_id

```
Cada comanda tiene un UUID único (transaction_id)
  → Si llega un mensaje con un tid que ya existe → se ignora
  → No importa cuántas veces llegue → se aplica UNA vez
```

4.2 Ventana temporal de dedup (comandaTidRecent)

```
tid marcado como "visto" por 45-120 segundos
  → Si llega un mensaje con tid visto recientemente → se ignora
  → Evita re-procesar echoes y reenvíos
```

4.3 Echo detection (comandaInsertLikelyOwnEcho)

```
Si el mensaje viene del mismo dispositivo que lo envió → se ignora
  → Evita que el emisor "reciba de vuelta" su propia comanda
```

4.4 Estado de comanda (no reaplica si ya está en ese estado)

```
Si comanda ya está en "entregada" → no se reaplica cambio a "pendiente"
  → Evita regresiones de estado
```

4.5 Digest de comandas (verificación de integridad)

```javascript
// CrozzoLanOpsSync.js:392
function localComandasDigest() {
  var list = global.comandas || [];
  // Hash de IDs + estados + count
  // Si el hash cambia → hay cambios → pull
  // Si el hash no cambia → no hay cambios → skip
}
```

Problema latente: El hash solo incluye IDs+count, no estado. Si solo cambia el estado de una comanda, el hash no cambia → pull se omite.

Fix propuesto: Incluir estado en el hash.

4.6 Pull completo al reconectar

```
Cuando un dispositivo se reconecta:
  1. Pull completo (GET /api/comandas?since=lastSync)
  2. Para cada comanda recibida:
     → Buscar por transaction_id
     → Si no existe → agregar
     → Si existe → actualizar (si es más reciente)
  3. Marcar como sincronizado
```

---

5. Escenarios de fallo y cómo se manejan

5.1 ¿Qué pasa si un mensaje se pierde?

```
Dispositivo A envía comanda → red se cae → mensaje no llega
        │
        ├──→ A guarda en outbox (localStorage)
        ├──→ A reintenta cada X segundos
        └──→ Cuando red vuelve → A reenvía → B recibe → agrega
```

No se pierde. El outbox persiste y reintenta.

5.2 ¿Qué pasa si un mensaje llega duplicado?

```
A envía comanda → B recibe → agrega → marca como visto
```

```
A reenvía (por timeout) → B recibe → busca transaction_id → ya existe → ignora
```

No se duplica. La dedup por transaction_id lo evita.

5.3 ¿Qué pasa si un dispositivo se cae y vuelve?

```
A se cae → B, C siguen funcionando
A vuelve → pull completo → recibe cambios que se perdió
  → Agrega los nuevos (por transaction_id)
  → Ignora los que ya tiene
  → Actualiza los que cambiaron (por timestamp)
```

No se pierde información. El pull completo recupera todo.

5.4 ¿Qué pasa si dos dispositivos editan la misma comanda?

```
A y B editan comanda "abc-123" simultáneamente
  → A envía cambio (estado: preparando, timestamp: T1)
  → B envía cambio (estado: listo, timestamp: T2)
  → Ambos reciben → merge por timestamp
  → T2 > T1 → estado "listo" gana
```

Merge por timestamp (LWW). El cambio más reciente gana.

5.5 ¿Qué pasa si hay 20 dispositivos?

```
20 dispositivos en la misma red
  → mDNS descubre todos
  → Cada uno envía comandas a todos (broadcast)
  → Cada receptor deduplica por transaction_id
  → No se duplican (transaction_id único)
  → Volumen: 20 × 6-12 comandas/hora = 120-240 comandas/hora
  → Tamaño: 120-240 × 1,500 bytes = 180-360 KB/hora
  → Ancho de banda: 50-800 Mbps → ✅ Sobrado
```

20 dispositivos no es un problema. El volumen es mínimo.

---

6. Riesgos reales (bugs latentes)

#	Bug	Causa	Impacto	Fix
1	Digest LAN ciego	Hash solo IDs+count	Pull omitido si solo cambia estado	Incluir estado en hash
2	WS broadcast panic	Handle::block_on en async	WS push puede crashear	Async await
3	Body 64KB truncation	Una sola lectura	Payloads grandes truncados	Read loop
4	Outbox sin idempotencia	Sin consecutivo	Duplicados tras 6h	Agregar idempotency key
5	Multi-writer carrito	Sin lock fuerte	Carrito confundido	Lock optimista

---

7. Resumen: cómo funciona la consistencia

```
1. Cada comanda tiene un UUID único (transaction_id)
2. Cada mensaje se marca como "visto" por 45-120s (comandaTidRecent)
3. Echo propio se detecta y se ignora (comandaInsertLikelyOwnEcho)
4. Duplicados se ignoran (findComandaForCloudPay por transaction_id)
5. Estado no regresa (si ya está "entregada", no vuelve a "pendiente")
6. Merge por timestamp (el cambio más reciente gana)
7. Pull completo al reconectar (recupera todo lo perdido)
8. Outbox + ACK (no se pierde información)
9. Persistencia local (SQLite/localStorage)
```

Resultado: Una comanda se aplica UNA vez en cada dispositivo, sin importar cuántas veces llegue o cuántos dispositivos estén en la red.

---

8. Lo que falta para ser 100% robusto

1. **Fix digest LAN** (incluir estado en hash) → evita pull omitido
2. **Fix WS panic** (async await) → evita crash de WS push
3. **Agregar idempotency key** a todas las operaciones → evita duplicados
4. **Lock optimista** en carrito → evita conflictos multi-writer
5. **Cifrado en LAN** → evita interceptación de datos

Con estos fixes, el sistema será robusto para 2-20 dispositivos en la misma red.